LAB 1: Synthesis và standard cells



1 Mục tiêu bài lab

Sau bài lab, hojc viên phải giải thích được vì sao hai lần tổng hợp cùng một RTL có thể tạo ra các tập standard cell khác nhau, đồng thời phân biệt việc định hướng mapper với việc ép một cell cụ thể.

2 Câu hỏi trung tâm

ABC chọn cell dựa vào đâu và người thiết kế có thể tự chọn cell hay không

Câu trả lời ngắn: ABC ánh xạ các mạng logic tổ hợp sang những cell hợp lệ trong Liberty. Cost chính thường là area và delay theo script đang chạy. Kết quả còn chịu ảnh hưởng bởi delay target, driving cell, output load, cell bị đánh dấu dont_use (không dùng) và cấu trúc logic sau optimization. Người thiết kế có thể định hướng hoặc giới hạn lựa chọn. Muốn bảo đảm đúng một cell cụ thể, có thể instantiate trực tiếp cell đó, nhưng thiết kế sẽ phụ thuộc công nghệ và tool mất bớt quyền tối ưu.

3 ABC thực sự nhìn thấy gì

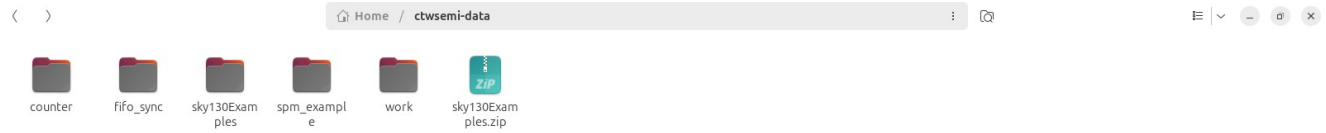
Thông tin	ABC sử dụng như thế nào
Boolean function	Xác định cell hoặc tổ hợp cell nào có thể thực hiện cùng chức năng, có thể tự tối ưu hàm boolean.
Area	Là diện tích của cell trong Liberty và được dùng khi tối ưu area.
Timing arcs	Ước lượng delay qua cell theo input, output và slew hoặc load.
Pin capacitance	Ảnh hưởng tải mà tầng trước phải điều khiển.
Driving cell	Mô hình hóa khả năng lái của nguồn tại đầu vào.
Output load	Mô hình hóa điện dung mà đầu ra phải điều khiển
Allowed cell set	Cell dont_use hoặc cell không có trong library sẽ không được chọn

ABC ở bước synthesis chưa biết wirelength (chiều dài dây dẫn) và parasitic (ký sinh của tụ, trở) sau PnR một cách chính xác. Do đó cell được chọn là nghiệm tốt theo mô hình trước layout, không phải lời giải signoff cuối cùng. Sau đó OpenROAD vẫn có thể resize cell, thay cell tương đương hoặc chèn buffer để sửa timing, capacitance và fanout.

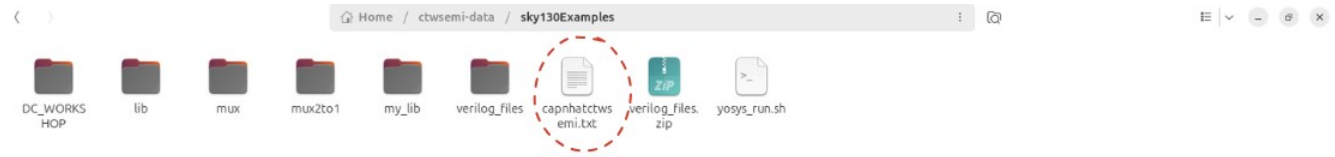
ABC mặc định không phải bộ tối ưu power hoàn chỉnh. Liberty có dữ liệu power, nhưng mapping thông thường chủ yếu tối ưu area và timing. Không nên kết luận rằng netlist có area nhỏ nhất cũng có power thấp nhất.

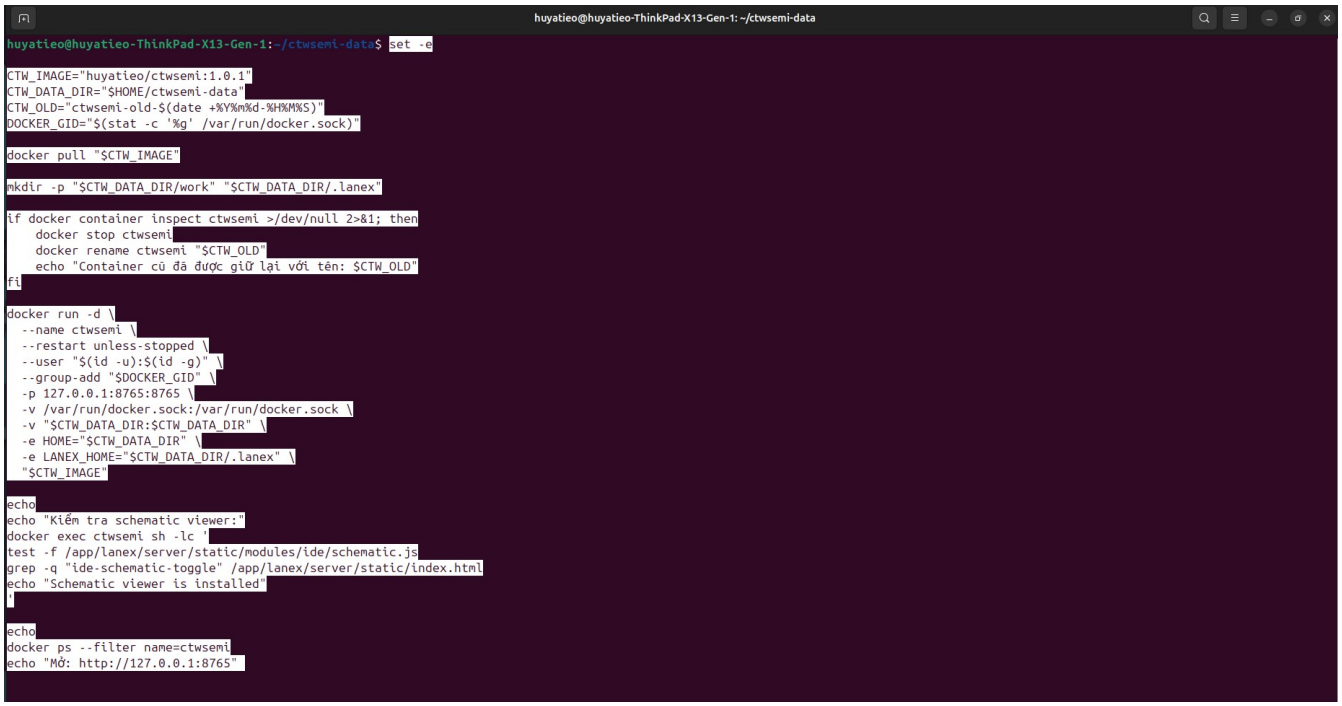
4 Chuẩn bị môi trường

1. Tải thư mục sky130xamples.zip về máy, giải nén vào thư mục ctwsemi-data.

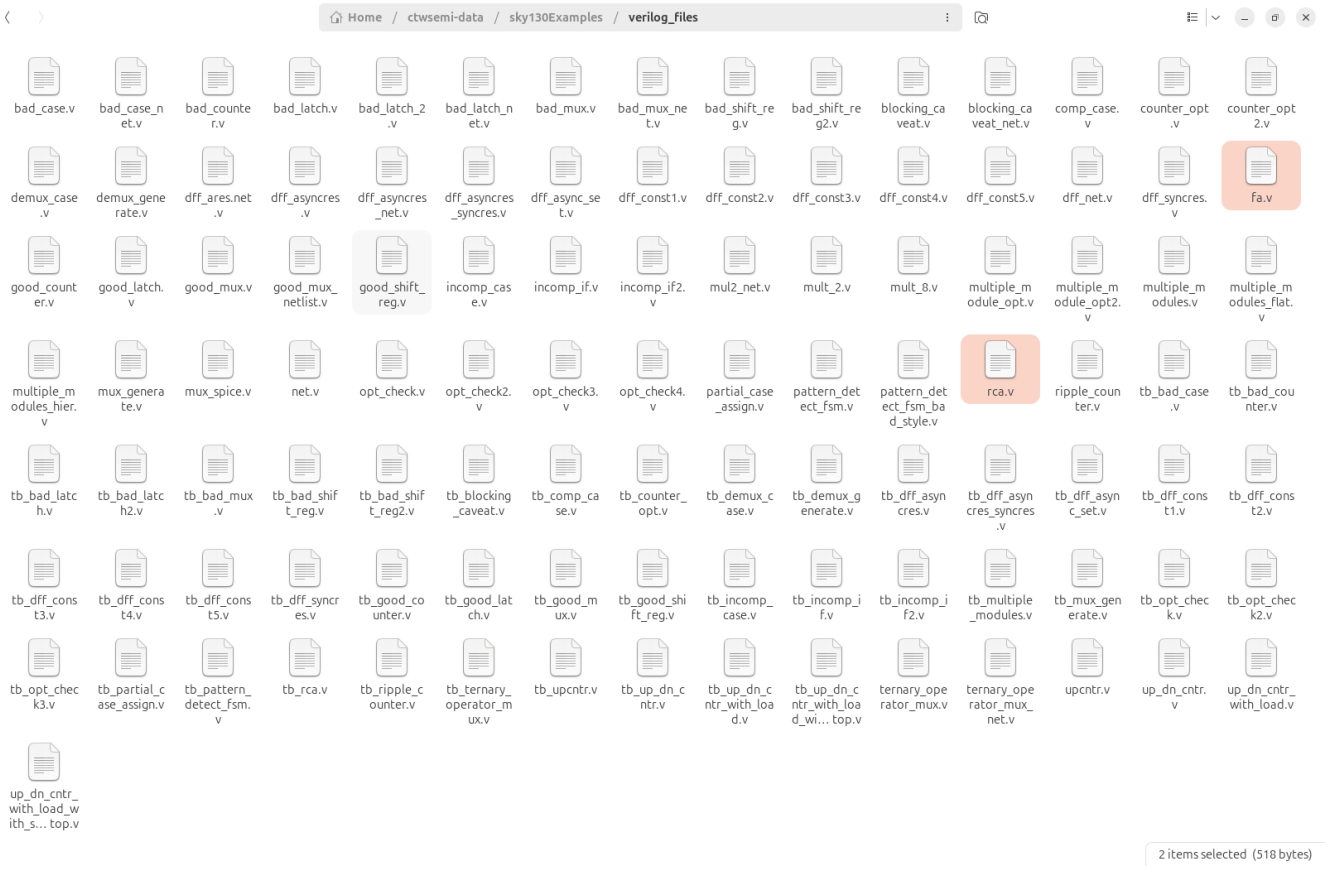


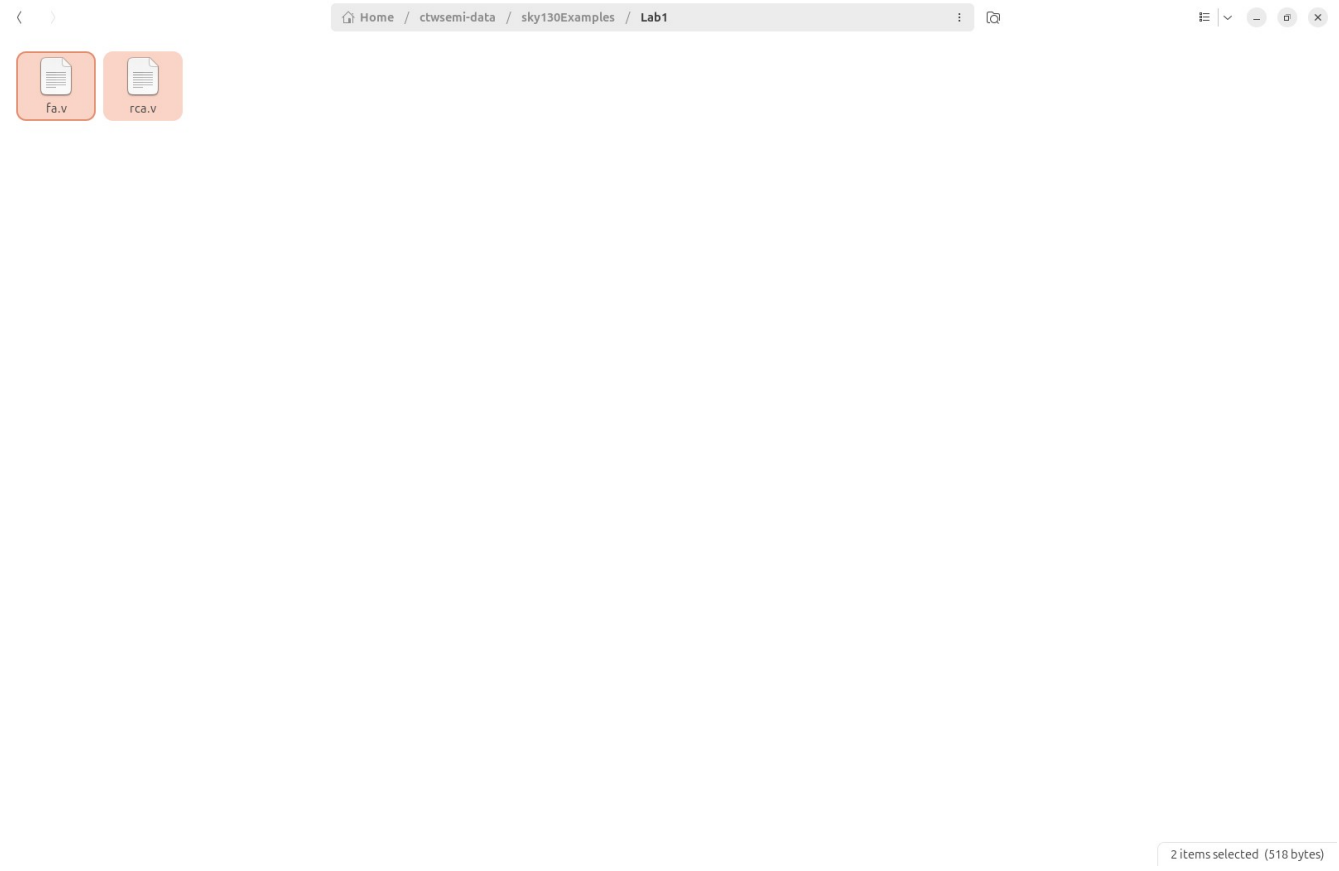
2. Vào thư mục Sky130Examples → Mở file **capnhatctwsemi.txt**, ctrl + A toàn bộ nội dung và copy vào cửa sổ dòng lệnh trên Ubuntu WSL.



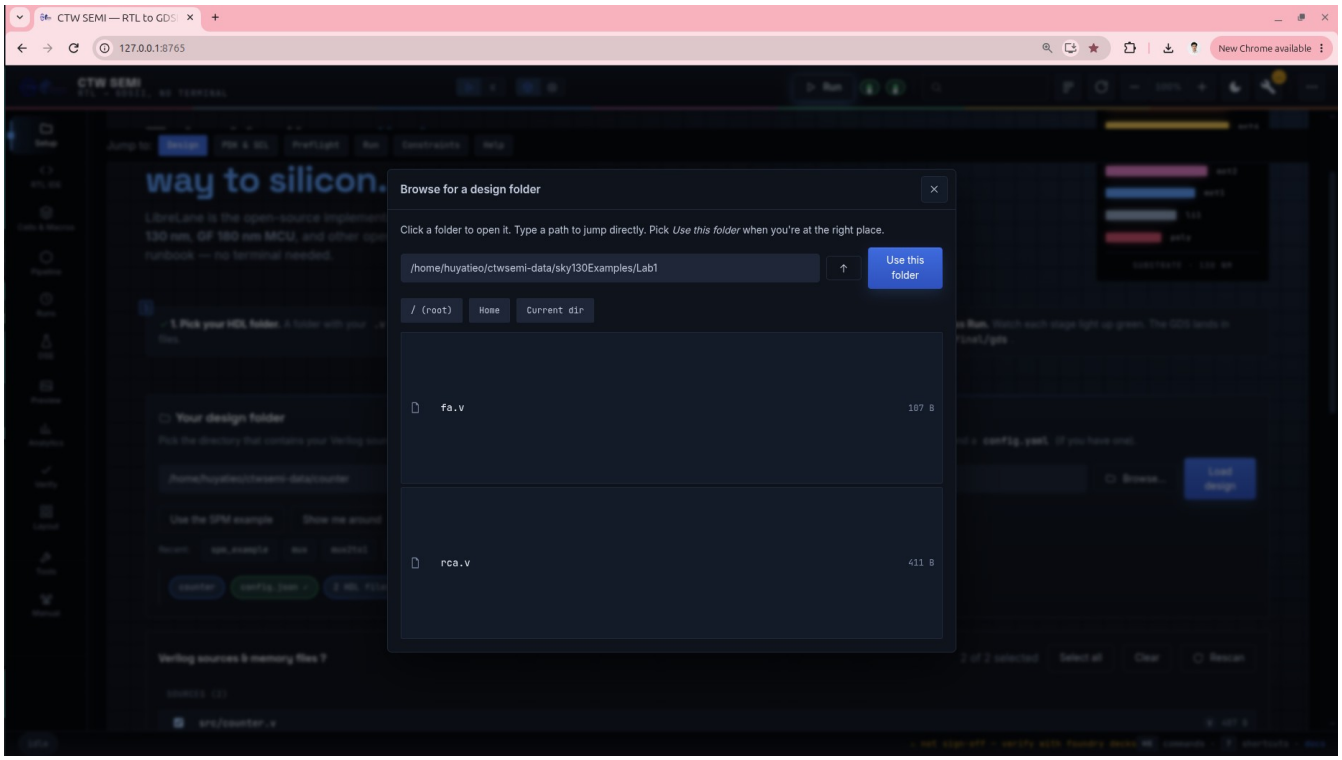


3. Tạo thư mục tên **Lab1**, cấu trúc thư mục bên trong chứa 2 file rca.v và fa.v, 2 file này nằm trong thư mục **verilog_files**

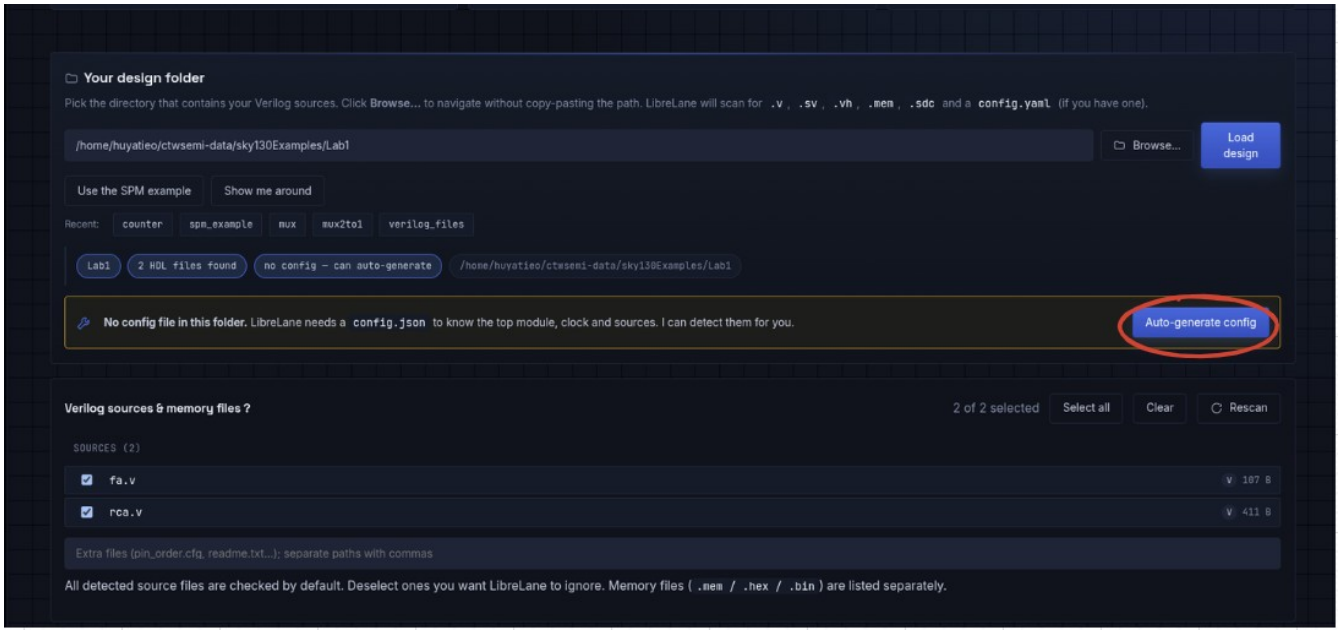

Home / ctwsemi-data / sky130Examples / verilog_files
bad_case.v
bad_case_net.v
bad_counter.v
bad_latch.v
bad_latch_2.v
bad_latch_net.v
bad_mux.v
bad_mux_net.v
bad_shift_reg.v
bad_shift_reg2.v
blocking_caveat.v
blocking_caveat_net.v
comp_case.v
counter_opt.v
counter_opt2.v
demux_case.v
demux_generate.v
dff_ares.net.v
dff_asyncres.v
dff_asyncres_net.v
dff_asyncres_syncres.v
dff_async_set.v
dff_const1.v
dff_const2.v
dff_const3.v
dff_const4.v
dff_const5.v
dff_net.v
dff_syncres.v
fa.v
good_counter.v
good_latch.v
good_mux.v
good_mux_netlist.v
good_shift_reg.v
incomp_case.v
incomp_if.v
incomp_if2.v
mul2_net.v
mult_2.v
mult_8.v
multiple_module_opt.v
multiple_module_opt2.v
multiple_modules.v
multiple_modules_flat.v
multiple_modules_hier.v
mux_generate.v
mux_spice.v
net.v
opt_check.v
opt_check2.v
opt_check3.v
opt_check4.v
partial_case_assign.v
pattern_detect_fsm.v
pattern_detect_fsm_bad_style.v
rca.v
ripple_counter.v
tb_bad_case.v
tb_bad_counter.v
tb_bad_latch.v
tb_bad_latch2.v
tb_bad_mux.v
tb_bad_shift_reg.v
tb_bad_shift_reg2.v
tb_blocking_caveat.v
tb_comp_case.v
tb_counter_opt.v
tb_demux_case.v
tb_demux_generate.v
tb_dff_asyncres.v
tb_dff_asyncres_syncres.v
tb_dff_async_set.v
tb_dff_const1.v
tb_dff_const2.v
tb_dff_const3.v
tb_dff_const4.v
tb_dff_const5.v
tb_dff_syncres.v
tb_good_counter.v
tb_good_latch.v
tb_good_mux.v
tb_good_shift_reg.v
tb_incomp_case.v
tb_incomp_if.v
tb_incomp_if2.v
tb_multiple_modules.v
tb_mux_generate.v
tb_opt_check.v
tb_opt_check2.v
tb_opt_check3.v
tb_partial_case_assign.v
tb_pattern_detect_fsm.v
tb_rca.v
tb_ripple_counter.v
tb_ternary_operator_mux.v
tb_upcntr.v
tb_up_dn_cntr.v
tb_up_dn_cntr_with_load.v
tb_up_dn_cntr_with_load_wi... top.v
ternary_operator_mux.v
ternary_operator_mux_net.v
upcntr.v
up_dn_cntr.v
up_dn_cntr_with_load.v
up_dn_cntr_with_load_with_s... top.v
2 items selected (518 bytes)

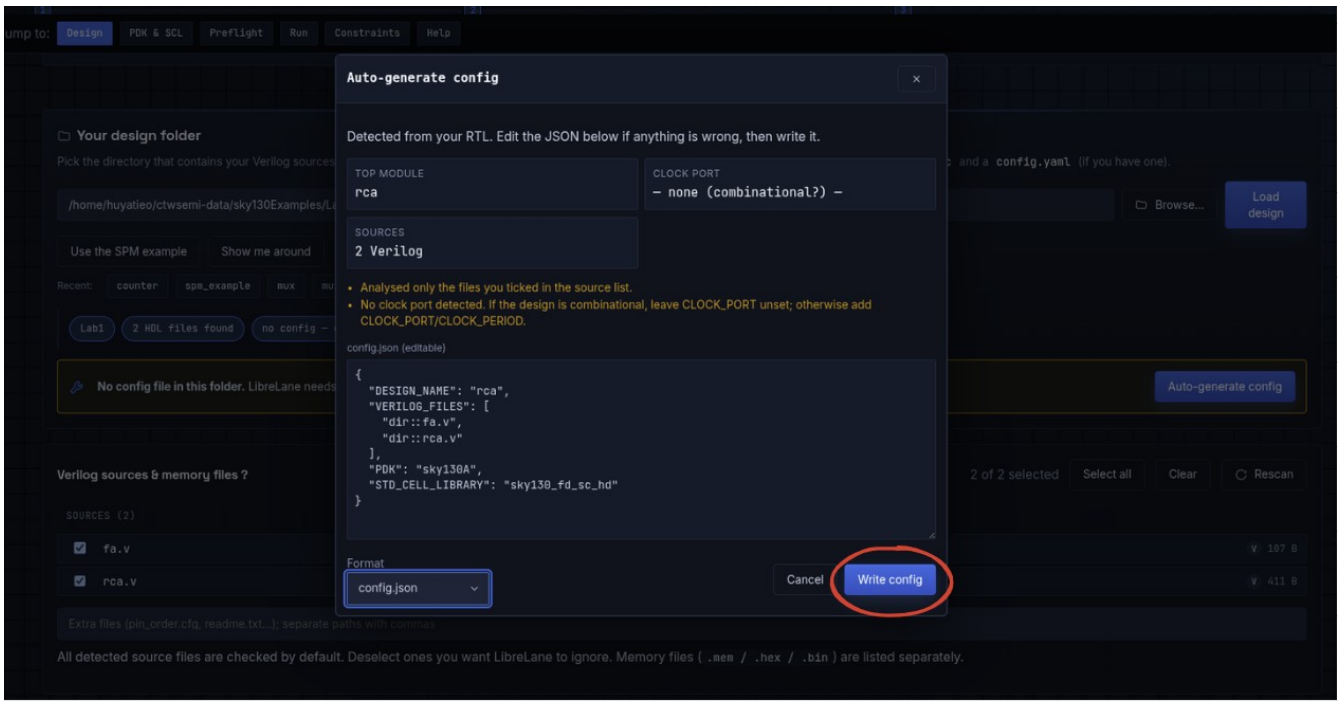


4. Khởi động CTW SEMI, vào Setup → Browse. Trỏ tới đường dẫn Lab1. Kiểm tra thấy đầy đủ 2 file như trong hình. Chọn Use this folder.

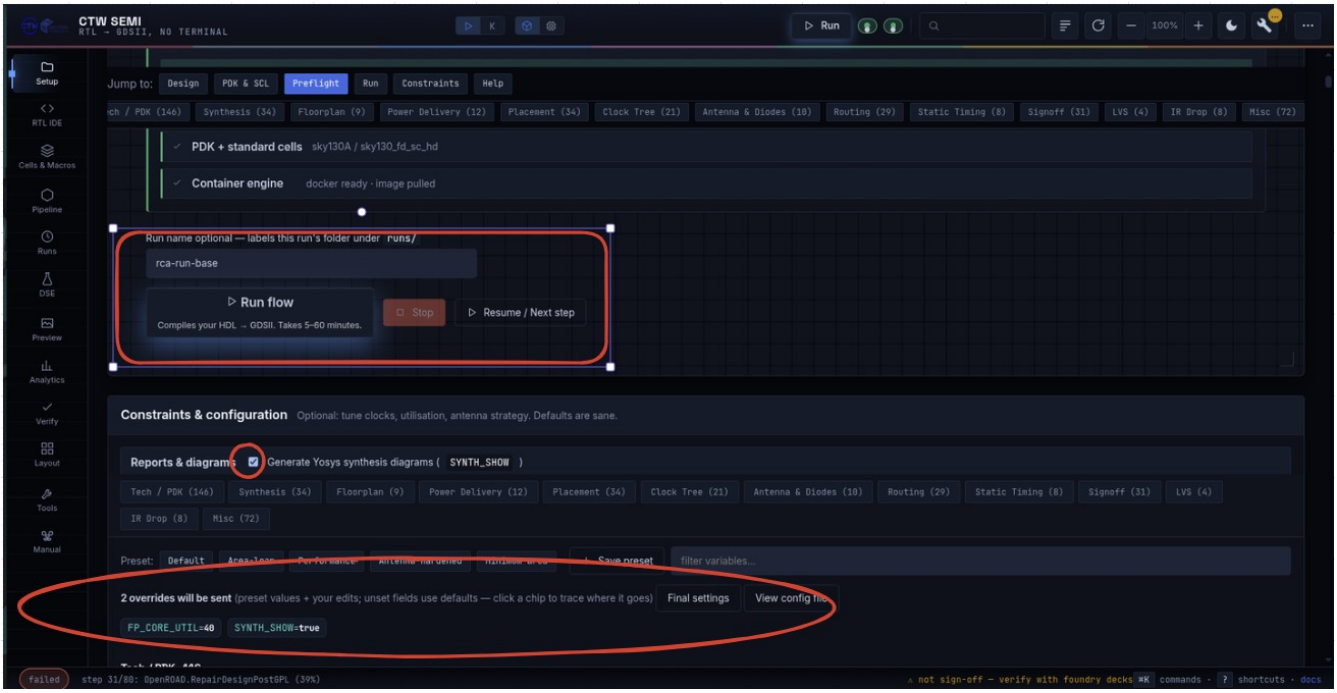


5. Chọn Auto-generate config, để tool tự tạo file cấu hình mặc đinh.





6. Đặt tên cho run, chạy Run flow lần đầu, sửa như Lab0, chọn FP_CORE_UTIL = 40, chọn SYNTH_SHOW, chạy 1 mạch không báo lỗi tới khi ra là GDS là ok.



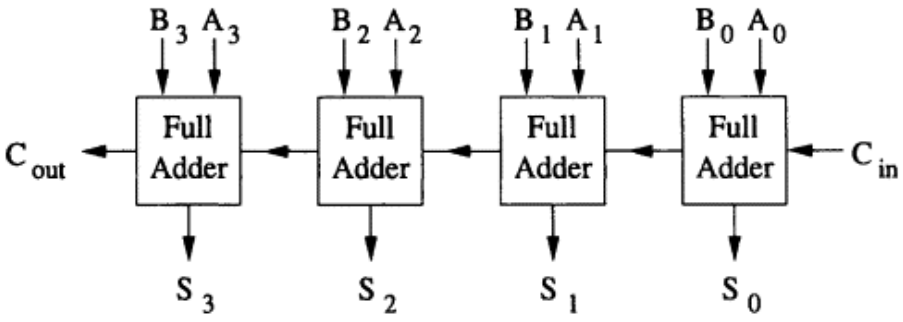
5 Khảo sát Ripple Carry Adder

1. Chức năng của bộ cộng RCA (Ripple Carry Adder). Đây là một mạch điện tử số, thực hiện chức năng cộng. Được cấu tạo bởi các bộ cộng toàn phần FA (Full Adder). Các bit nhớ (bit carry) được lan truyền vào trong mạch từ Cin tới Cout. Bộ cộng phía sau ở vị trí bit cao hơn phải chờ cho kết quả ở bit trước đó tính xong.

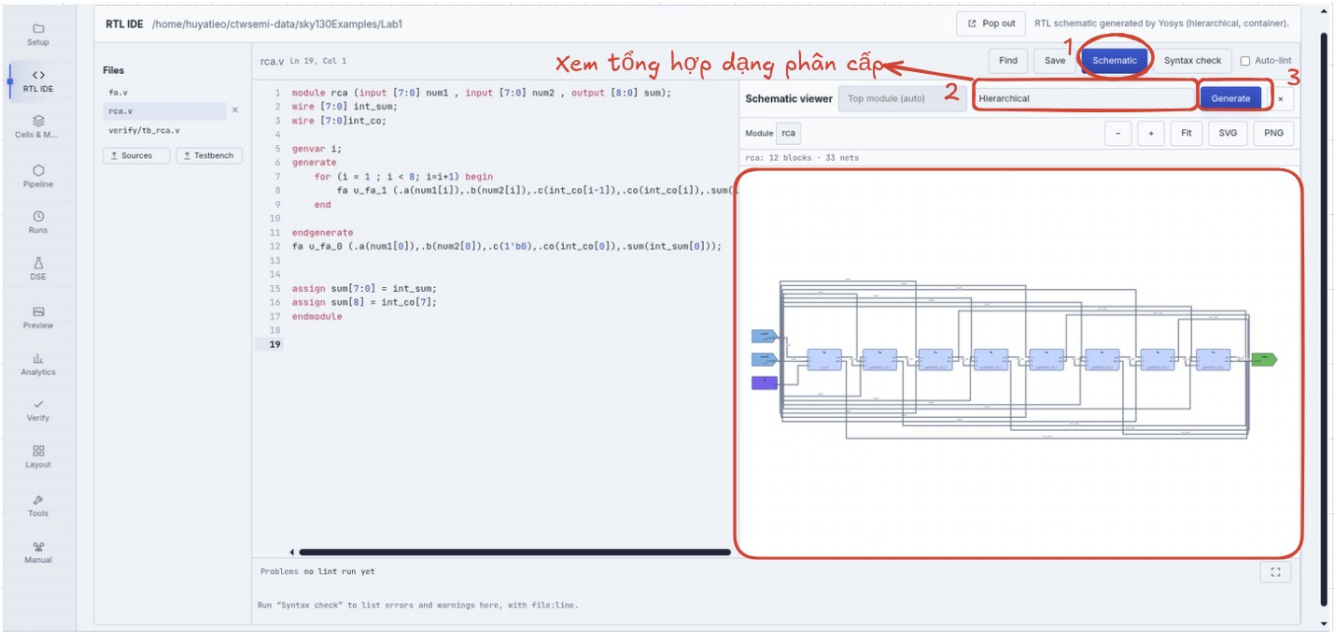
Ripple carry adder

The ripple carry, or parallel adder, arises out of considering how we perform addition, and is therefore a heuristic solution. Two numbers can be added by beginning with the two least significant digits to produce their sum, plus a carry-out bit (if necessary). Then the next two digits are added (together with any carry-in bit from the addition of the first two digits) to produce the next sum digit and any carry-out bit produced at this stage. This process is then repeated until the most significant digits are reached (see Section 2.5.1).

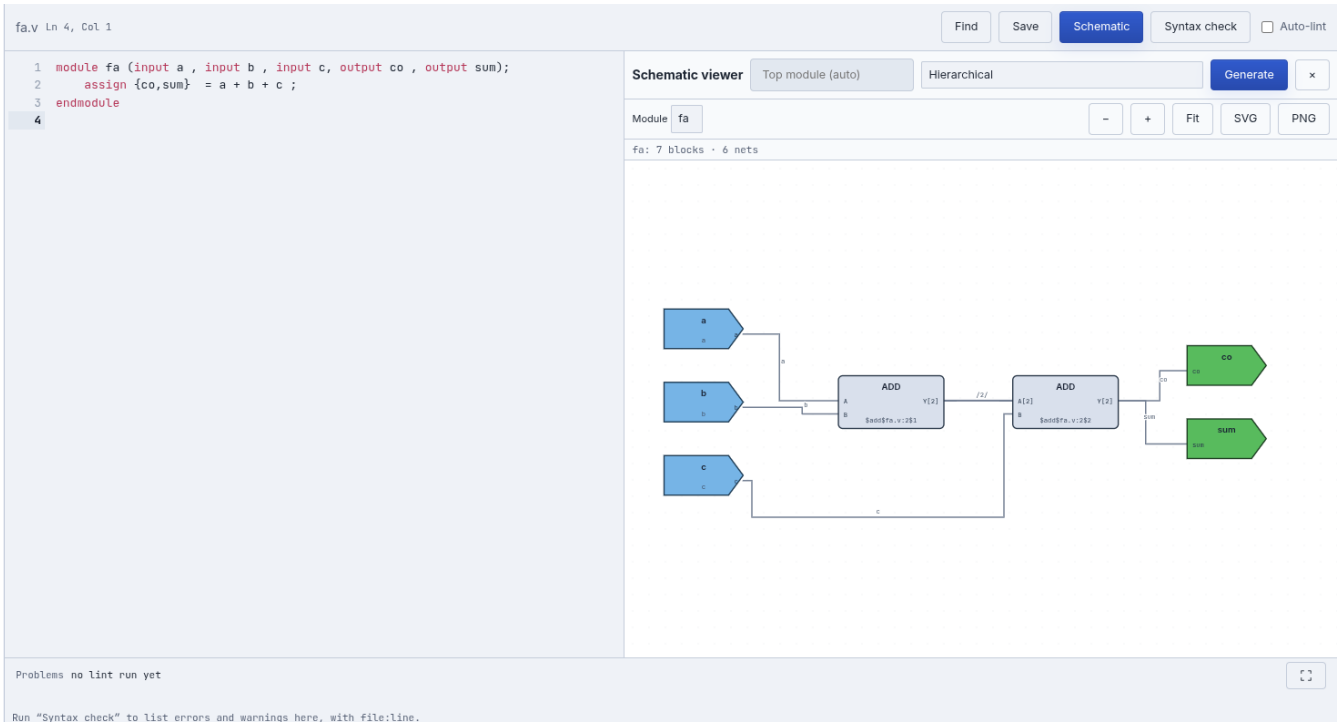
To implement this procedure, for binary arithmetic, what is required is a logic block which can take two input bits, and add them together with a carry-in bit to produce their sum and a carry-out bit. This is exactly what the full adder, described earlier in Section 4.1.5, does. Consequently by joining four full adders together, with the carry-out from one adder connected to the carry-in of the next, a four-bit adder can be produced. This is shown in Fig. 4.15.



2. Mở RTL IDE, chọn biểu Schematic để xem dạng tổng hợp mạch của rca.v

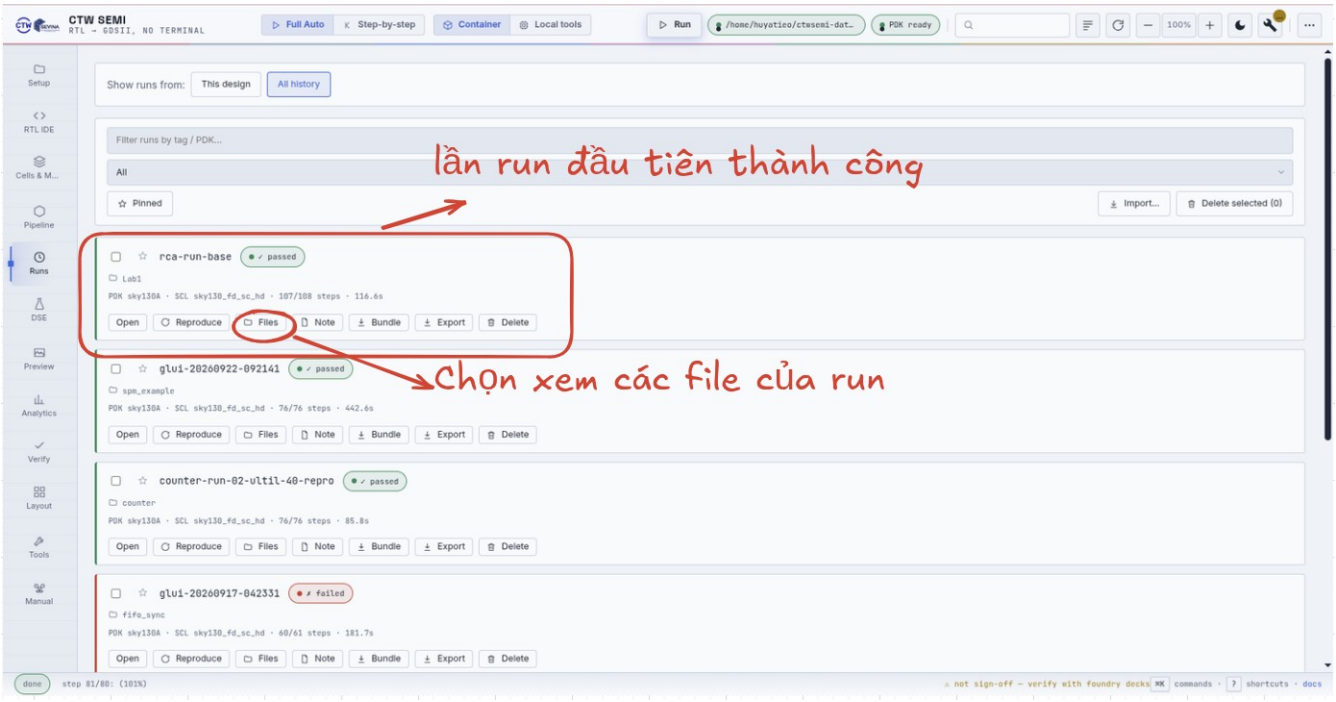


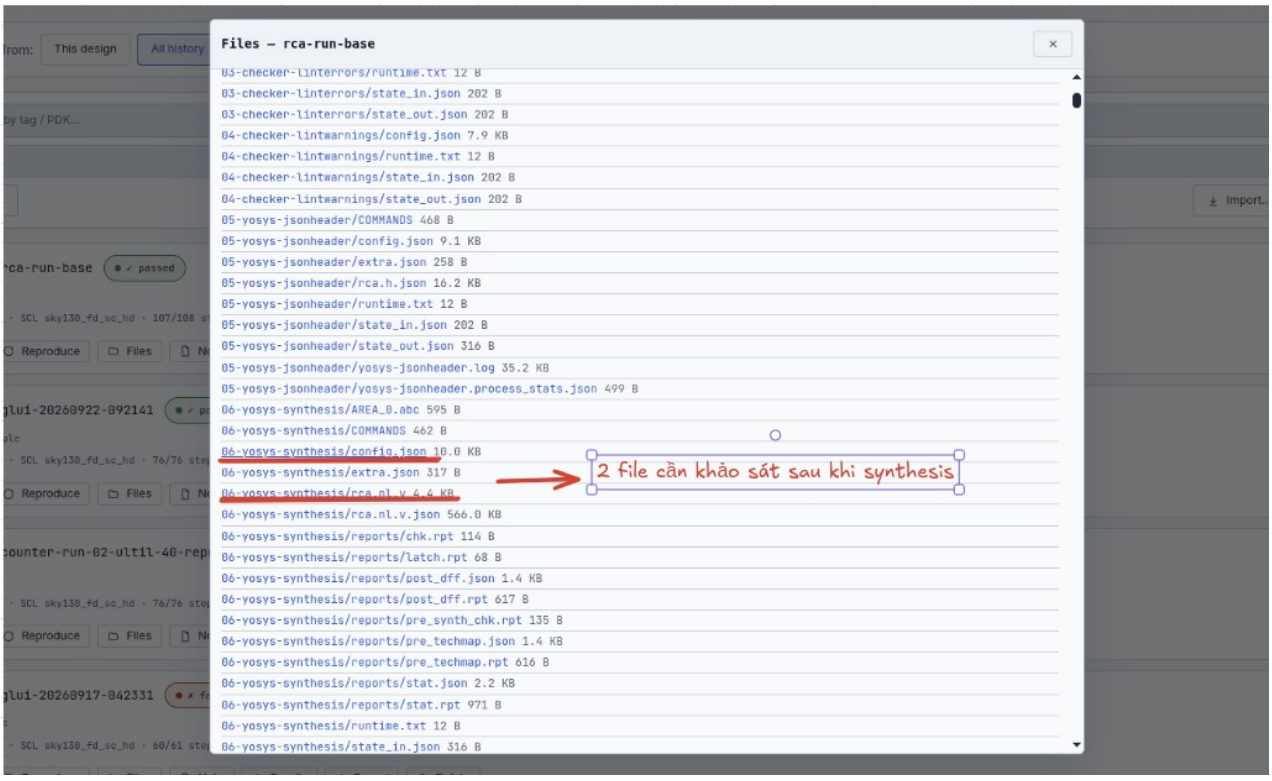
Nhận xét mạch tổng hợp: Đây là mạch cộng rca cho 2 số có đầu vào 8 bit là num1 và num2, kết quả của phép cộng là sum 9 bit (gồm 8 bit kết quả và 1 bit carry ở MSB). Do đó số lượng các bộ fa (Full Adder) được tạo ra là 8, tương ứng cho 8 bit đầu vào. Mỗi bộ fa nhận vào 1 bit đầu vào a, 1 bit đầu vào b, 1 bit carry đầu vào c, kết quả tạo ra 1 bit carry đầu ra co, 1 bit kết quả sum.



6 Thí nghiệm 1

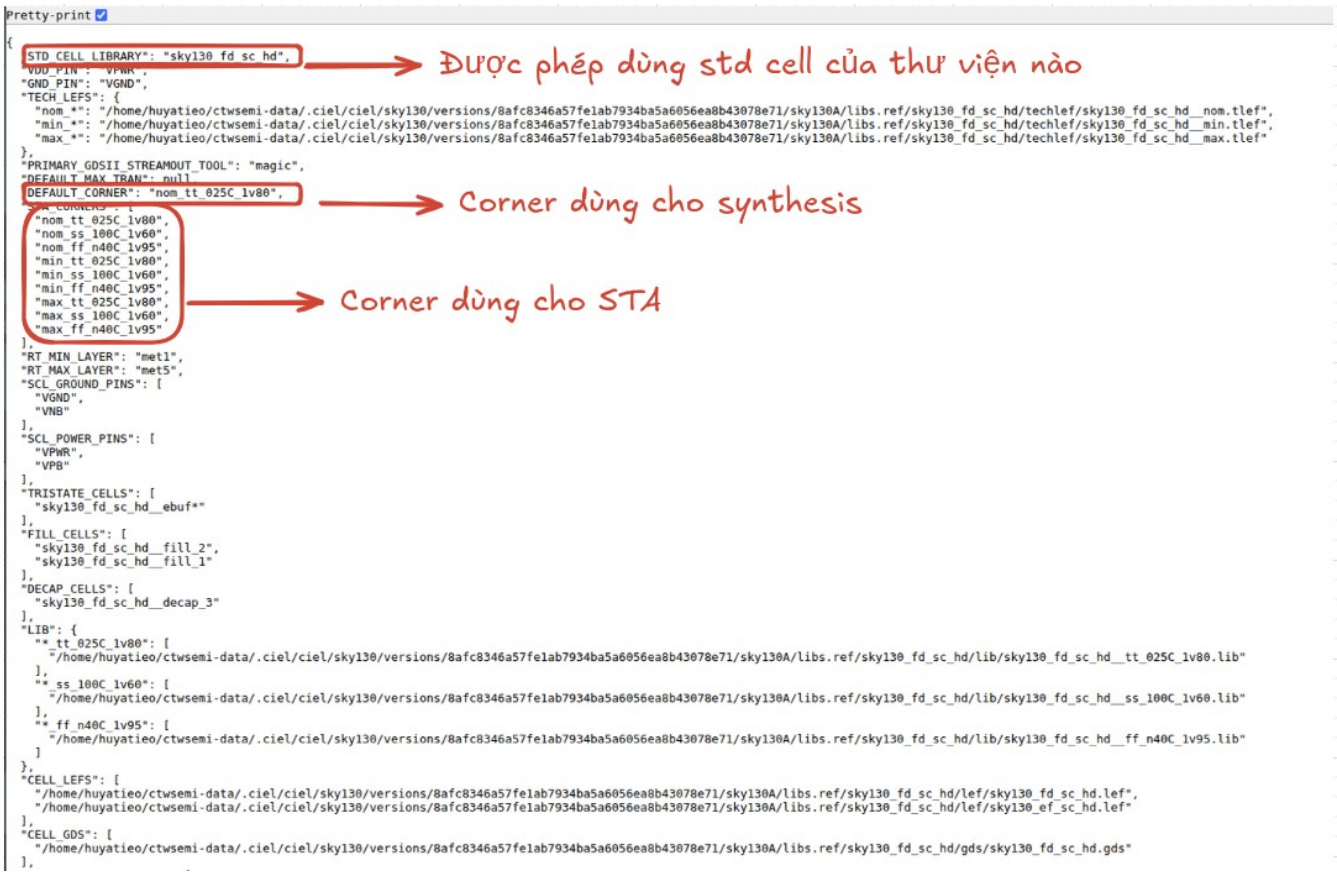
Ở lần chạy đầu tiên: rca-run-base, ta bấm vào runs → chọn đúng lần chạy này để xem các cấu hình mà ABC nhận được để chọn cell và các chiến thuật tổng hợp.





4. Config.json cho ta biết cấu hình mặc định của librelane cho synthesis. Cấu hình này được ghi đè khi có sự thay đổi trong CTW SEMI (configuration & constraint), hoặc tự viết config.json cho dự án.

   Mở vào bên trong config.json



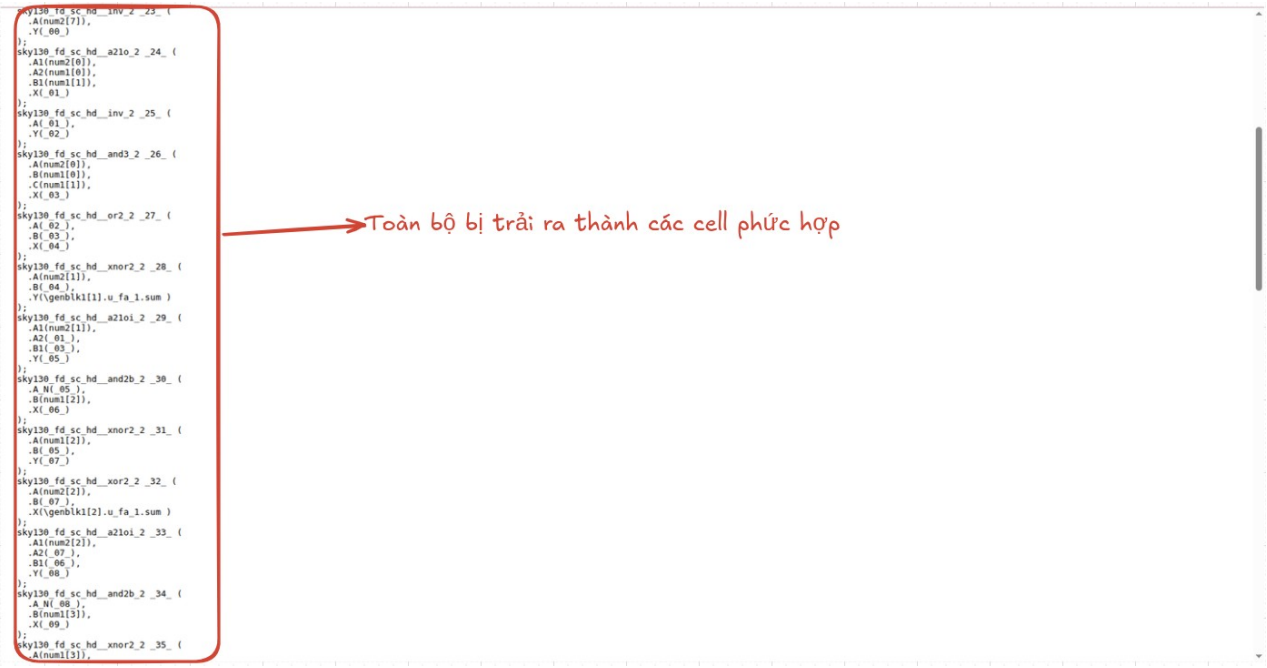
Khi synthesis, tool Yosys tự chọn 1 corner trong 9 corner dùng cho STA để đại diện và chọn các cell để tổng hợp. Nhưng sau này, khi xét STA thì cả 9 corner đều phải pass thì chip mới chạy. Tiếp tục, ta xem các quyết định quan trọng, như hình dưới đây.



- ý nghĩa các biến quan trọng.

Tên biến	Tham số	Ý nghĩa
SYNTH_STRATEGY	AREA 0	ABC dùng strategy thiên về diện tích
SYNTH_HIERARCHY_MODE	Flatten	Xóa nhòa ranh giới giữa các module phân cấp
SYNTH_SHARE_RESOURCE	True	Cho phép gộp logic dùng chung

5. Kiểm tra kết quả của ABC, chọn file rca.nl.v để xem kết quả tổng hợp dưới dạng các standard cell



Vì chọn SYNTH_HIERARCHY_MODE = “flatten”, do đó khi ta tìm sẽ không thấy các module con tên full adder trong file. Mặc dầu có con cell chuyên cho bộ full adder, nhưng ABC lại không dùng để tổng hợp, vì ranh giới giữa các module phân cấp như full adder đã bị xóa bỏ. ABC chọn các con cell khác thay thế. Dĩ nhiên vẫn đảm bảo chức năng y hệt như full adder.

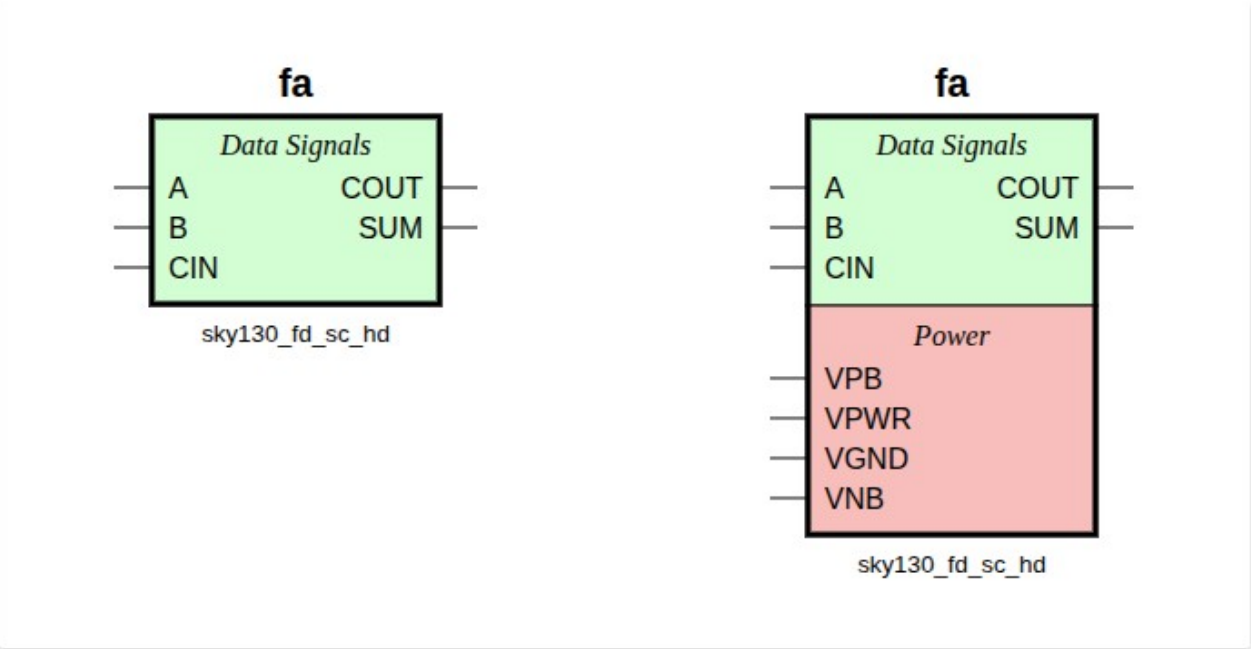
sky130_fd_sc_hd__fa

Full adder

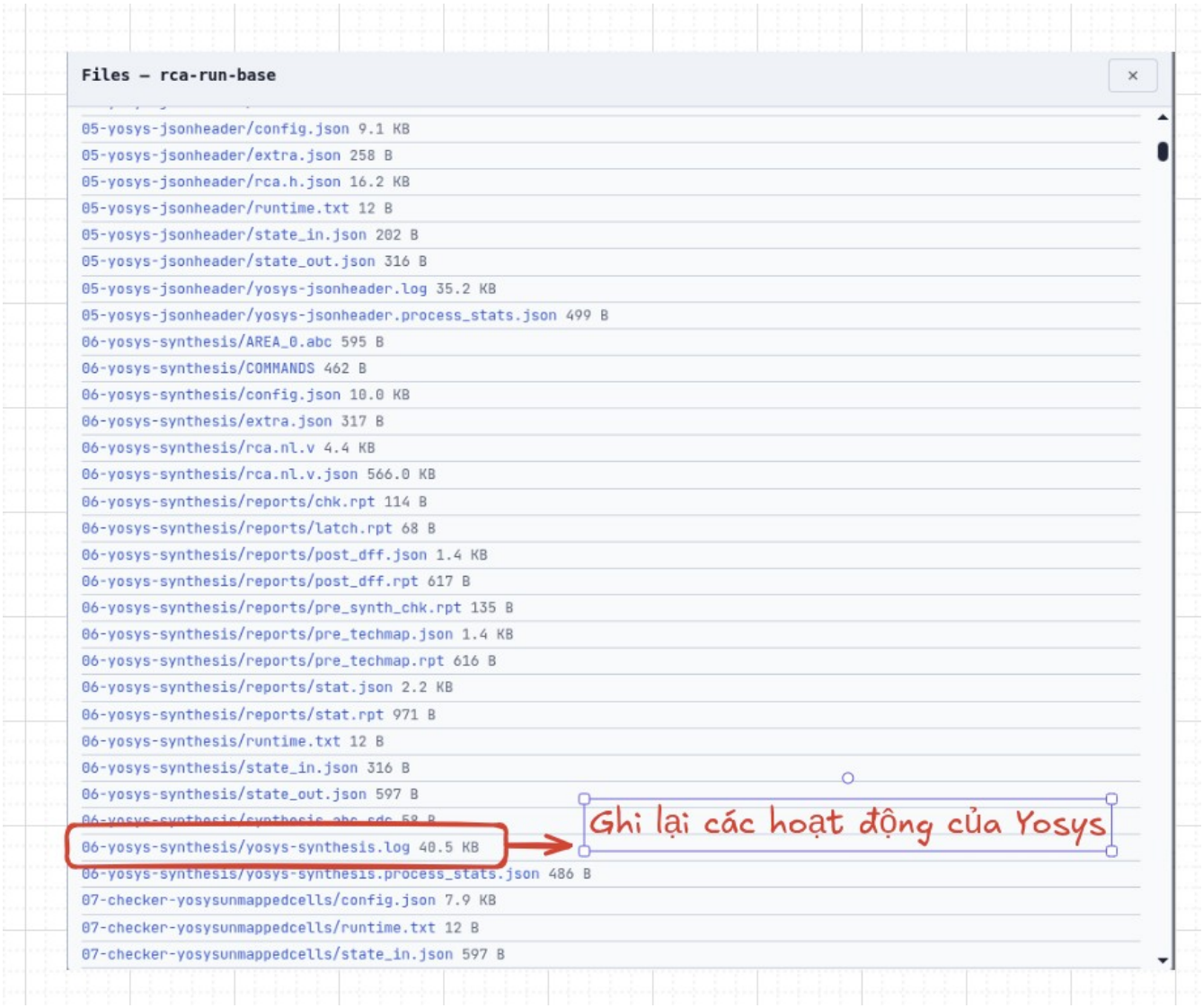
This is a stub of cell description file

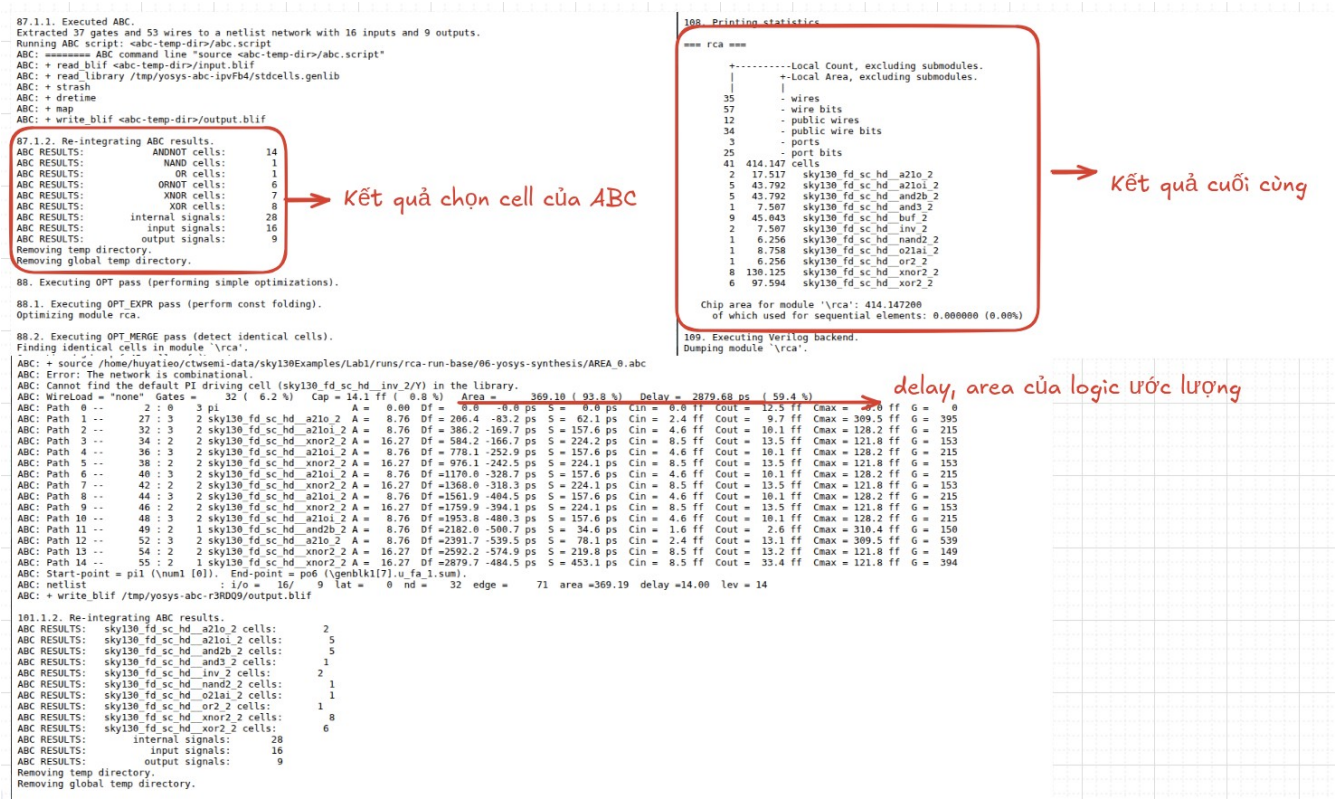
- **Cell name**: sky130_fd_sc_hd__fa
- **Type**: cell
- **Verilog name**: sky130_fd_sc_hd__fa
- **Library**: sky130_fd_sc_hd
- **Inputs**: 3 (A, B, CIN)
- **Outputs**: 2 (COUT, SUM)

Symbols ¶



6. Tiếp đến, ta mở file yosys-synthesis.log, để xem kết quả của ABC như các loại standard cell, tổng số cell của thiết kế, diện tích,...


Files – rca-run-base
05-yosys-jsonheader/config.json 9.1 KB
05-yosys-jsonheader/extra.json 258 B
05-yosys-jsonheader/rca.h.json 16.2 KB
05-yosys-jsonheader/runtime.txt 12 B
05-yosys-jsonheader/state_in.json 202 B
05-yosys-jsonheader/state_out.json 316 B
05-yosys-jsonheader/yosys-jsonheader.log 35.2 KB
05-yosys-jsonheader/yosys-jsonheader.process_stats.json 499 B
06-yosys-synthesis/AREA_0.abc 595 B
06-yosys-synthesis/COMMANDS 462 B
06-yosys-synthesis/config.json 10.0 KB
06-yosys-synthesis/extra.json 317 B
06-yosys-synthesis/rca.nl.v 4.4 KB
06-yosys-synthesis/rca.nl.v.json 566.0 KB
06-yosys-synthesis/reports/chk.rpt 114 B
06-yosys-synthesis/reports/latch.rpt 68 B
06-yosys-synthesis/reports/post_dff.json 1.4 KB
06-yosys-synthesis/reports/post_dff.rpt 617 B
06-yosys-synthesis/reports/pre_synth_chk.rpt 135 B
06-yosys-synthesis/reports/pre_techmap.json 1.4 KB
06-yosys-synthesis/reports/pre_techmap.rpt 616 B
06-yosys-synthesis/reports/stat.json 2.2 KB
06-yosys-synthesis/reports/stat.rpt 971 B
06-yosys-synthesis/runtime.txt 12 B
06-yosys-synthesis/state_in.json 316 B
06-yosys-synthesis/state_out.json 597 B
06-yosys-synthesis/yosys-synthesis.log 40.5 KB
06-yosys-synthesis/yosys-synthesis.process_stats.json 486 B
07-checker-yosysunmappedcells/config.json 7.9 KB
07-checker-yosysunmappedcells/runtime.txt 12 B
07-checker-yosysunmappedcells/state_in.json 597 B
Ghi lại các hoạt động của Yosys



Nhận xét: ABC tuy báo cáo là 37 cells logic (14 + 1 + 1 + 6 + 7 + 8), nhưng kết quả cuối cùng lại báo 32 cells logic (41 cell – 9 cell sky130_fd_sc_hd_buf_2, đây là 9 con cell output tương ứng với 9 bit đầu ra). Lý do là vì ABC mapping chưa phải bước cuối cùng. Sau đó Yosys làm 1 loạt các bước rút gọn, truyền tham số, tối ưu để chốt kết quả cuối cùng trong nl.v là 41 cell và tính ra diện tích sơ bộ. Thời gian delay là 2.8 ns (2879 ps), đây là thời gian trước khi PnR nên sẽ không chính xác, nhưng có thể làm điểm tham chiếu giữa các strategy khác nhau.

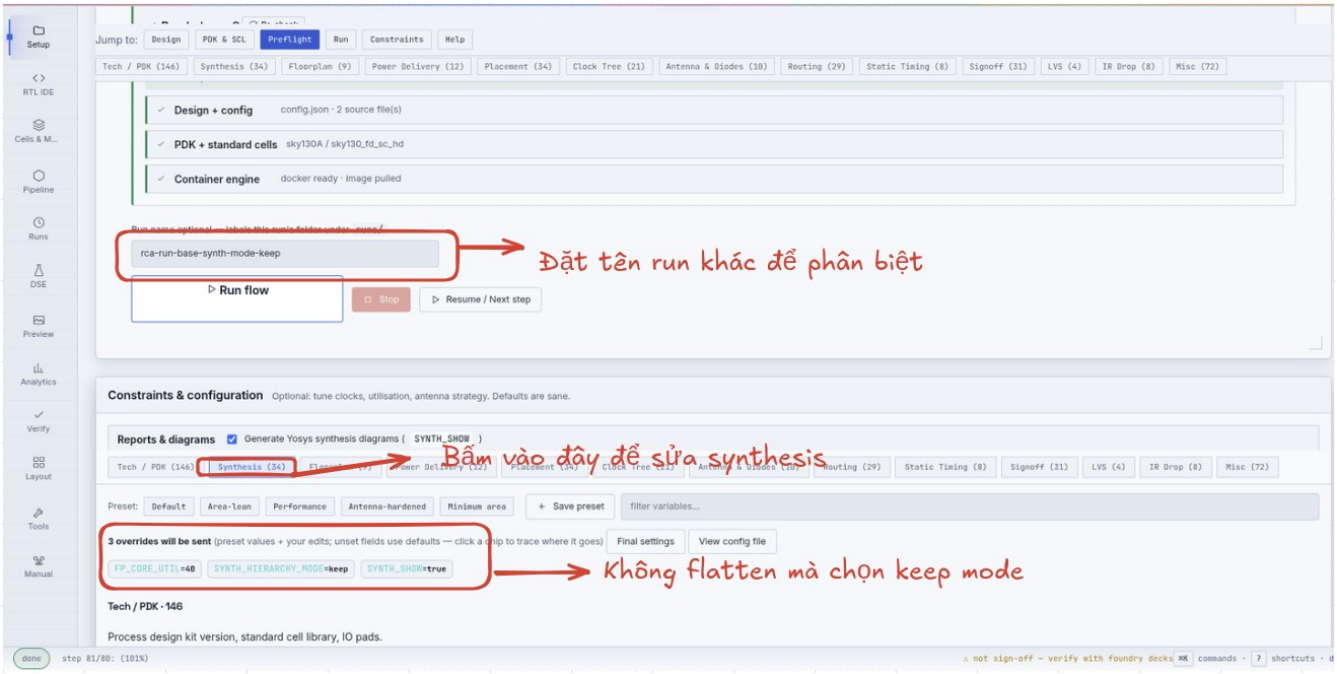
→ Synthesis bảo toàn hàm logic, không bảo toàn số cell trung gian. Muốn đảm bảo đúng. Ta thường thực hiện GLS (Gate level netlist simulation) để đảm bảo khớp chức năng giữa RTL netlist và GL netlist.

Thực hiện bảng báo cáo với các strategy khác nhau khi synthesis

Run	Số cell	Area	Delay ABC	Area ABC	Nhận xét
baseline					
SYNTH_STRATEGY = AREA 3					

Run	Số cell	Area	Delay ABC	Area ABC	Nhận xét
SYNTH_STRATEGY = DELAY 0					
SYNTH_HIERACHY_MODE = keep					

Ví dụ với cách chỉnh như hình dưới:



7 Thí nghiệm 2

Trong trường hợp ta không muốn tool chọn, mà muốn sử dụng chính xác con cell mà ta muốn. Thì làm như hình dưới:

RTL IDE /home/huyatieo/ctwsemi-data/sky130Examples/Lab1

Files

fa.v

rca.v

verify/tb_rca.v

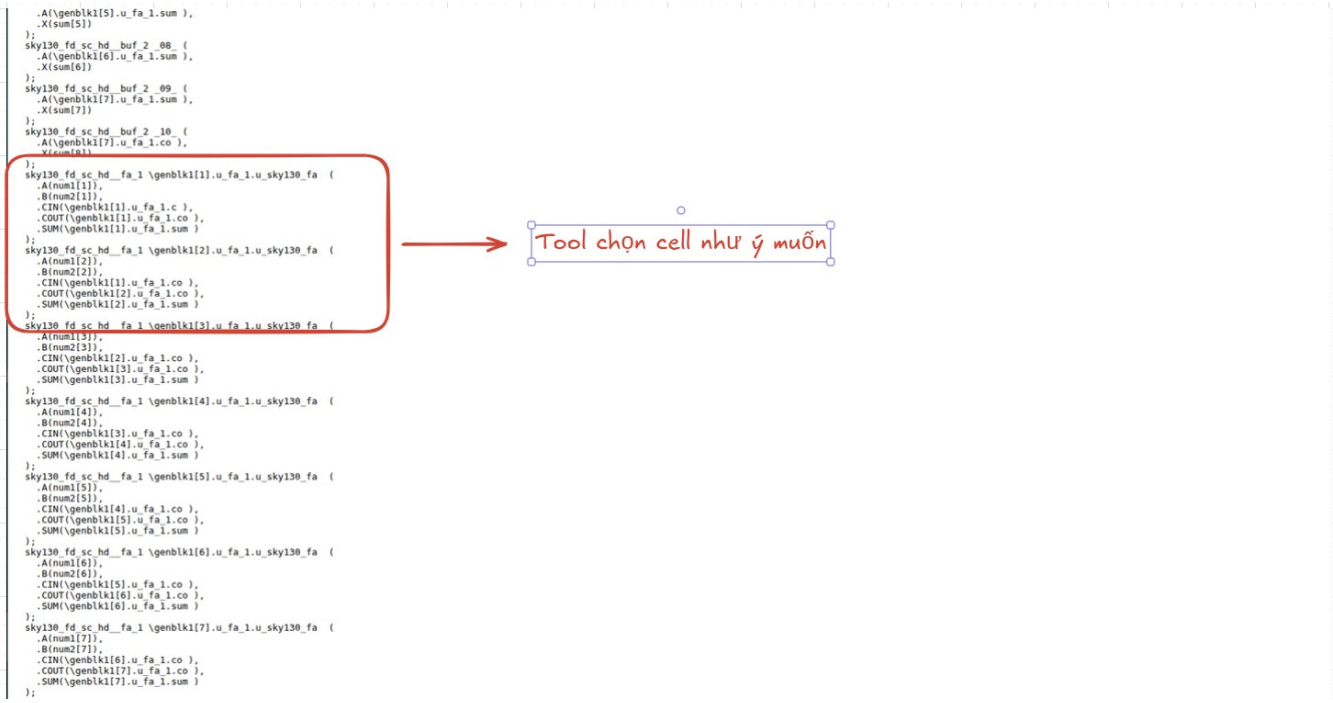
Sources | Testbench

fa.v Ln 19, Col 1 — Find | Save | Schematic | Syntax check | Auto-lint

```
1  module fa (
2      input  wire a,
3      input  wire b,
4      input  wire c,
5      output wire co,
6      output wire sum
7  );
8
9      (* keep *)
10     sky130_fd_sc_hd__fa_1 u_sky130_fa (
11         .A    (a),
12         .B    (b),
13         .CIN  (c),
14         .COUT (co),
15         .SUM  (sum)
16     );
17
18 endmodule
19
```

Lúc này ta đã khóa chặt tool vào duy nhất cho 1 công nghệ là sky130, không thể dùng cho các công nghệ khác. Cách viết này chỉ thực sự khuyến nghị khi ta hiểu rõ về đặc tính của cell.

7. Kết quả sau khi chạy:



Tài liệu tham khảo

- YosysHQ documentation. Technology mapping and abc command. https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/index_passes_techmap.html
- LibreLane documentation. Built in steps and synthesis configuration variables. https://librelane.readthedocs.io/en/latest/reference/step_config_vars.html
- LibreLane documentation. Synthesis Exploration flow. https://librelane.readthedocs.io/en/latest/reference/flows.html
- SkyWater PDK documentation. Standard cell libraries and process design kit information. https://skywater-pdk.readthedocs.io/